

Conventional Desktops: Hardening Candidates and Compatibility Costs

Conventional Desktops: Hardening Candidates and Compatibility Costs I

The two preceding sections examined the architectural poles of this review: hypervisor-based compartmentalization in sec. 5 and declarative, rebuildable operations in sec. 6. Most operators, however, will run a conventional Linux desktop for the foreseeable future, so the desktop candidates deserve the same property-based scrutiny rather than a distribution-label comparison. This section evaluates seven desktop-focused candidates against the nine properties of sec. 4, with particular attention to `application_confinement`, `integrity`, and `human_usability`: whether shipped defaults actually constrain applications, whether boot and update paths verify what runs, and whether a real operator will sustain the configuration. All judgments follow from documented designs, release notes, and advisories reviewed on 2026-09-10; none rests on a comparative penetration test, and none is

expressed as a numeric score.

The desktop lane is also where the fastest documented movement in this review is happening. The secureblue release sequence from v4.3.0 through v4.9.1, Fedora's removal of the GNOME X11 session, Debian's LTS planning, and Ubuntu's AppArmor and snap-permission changes all landed inside the review window, and each moves a candidate's documented defaults in a direction this section can now assess concretely rather than in principle (secureblue Project 2026b, 2026c; Fedora Project 2026e; Project 2025; Canonical 2026).

Conventional Desktops: Hardening Candidates and Compatibility Costs I

Table 1: Desktop candidates compared by documented security-relevant design, important limitation, and analytical assessment. Judgments are analytical, derived from documented designs and project documentation; they are not results of a comparative penetration test.

Candidate	Documented security-relevant design	Important limitation	Assessment
Fedora Silverblue / Kinoite	Silverblue documents read-only system paths with writable /etc and /var, and Fedora enforces SELinux as its default baseline (Fedora Project 2026b, 2026c). Fedora is also removing the GNOME X11 session: the Wayland-only GNOME change strips X11 session packages from the Fedora 43 and 44 GNOME offerings (Fedora Project 2026e).	Persistent writable state remains, and approximately 13 months of release support require regular upgrades to a new release, with Fedora 42 reaching end of life on 2026-05-13 and Fedora 43 on 2026-12-09 (Fedora Project 2026a). Silverblue-specific details should not be transplanted onto Kinoite without checking each image.	A credible atomic-desktop direction inside the Fedora ecosystem: image-based system updates, an enforcing MAC baseline by default, and a display stack being reduced to one code path. Its documented claims concern system paths and update mechanics, not containment of a compromised application's access to that application's own data.
Fedora Workstation	Fedora's documented baseline includes SELinux enforcement without choosing an atomic variant, and the same Wayland-only GNOME trajectory applies to the non-atomic desktop (Fedora Project 2026c, 2026e).	SELinux policy and application sandboxing must be evaluated as deployed; the existence of a kernel access-control mechanism does not by itself establish a tight sandbox for each application (Fedora Project 2026d). The X11 removal also narrows fallback options for specialized workflows that still require X11.	A defensible compatibility-first choice when the operator preserves the baseline and places dangerous work in separate isolation. An atomic variant does not automatically win every runtime-security comparison against it.

Conventional Desktops: Hardening Candidates and Compatibility Costs II

secureblue

The project documents a Fedora Atomic base, broad hardened_malloc deployment, a confined Trivalent browser, removal of SUID-root programs, restrictive application settings, and signed-container policy (secureblue Project 2026a). Since v4.3.0, an SELinux policy denies user-namespace creation to unconfined processes and the container domain by default — a default-deny posture with explicit ujust toggles for Flatpak, Chromium, and container workflows (secureblue Project 2026b). The v4.9.1 release completed a SUID-less userland on 2026-05-26, removing the legacy setuid privilege path structurally rather than by policy (secureblue Project 2026c).

Debian stable

Debian operates a coordinated security-advisory process and recommends unattended security upgrades; LTS extends stable support under a separate group from the (FSF) The same documentation records compatibility-affecting restrictions: disabled Xwayland by default, default-deny user namespaces that require per-domain toggles, and image renames that follow Fedora's own (the browser rebranded to Trivalent in the v4.4.0 cycle) (secureblue Project 2026a). These are project-documented measures, not measured superiority results.

A maintenance policy is not evidence of strong isolation between arbitrary desktop applications, and the two support phases are organized differently; conflating them misstates who maintains what. The

The most compelling conventional security-focused desktop candidate in this review, conditional on application compatibility, update reliability, and the operator not undoing the hardening to restore convenience.

A sound conservative platform when deliberately configured for the actual workload, now with the longest explicitly dated LTS horizon in this table. A well-maintained, narrowly configured installation is

Conventional Desktops: Hardening Candidates and Compatibility Costs III

Ubuntu LTS

Ubuntu documents five years of standard security maintenance for LTS releases, with longer coverage under specified offerings; snap confinement distinguishes strict, classic, and development modes (Ubuntu 2026b, 2026a). Ubuntu 26.04 LTS extends AppArmor enforcement to more daemons (OpenLDAP among them) and ships an experimental permission-prompting mechanism for strictly confined snaps (Canonical 2026).

Classic snaps carry no confinement, so “installed as a snap” is not sufficient evidence of application isolation; coverage must be matched to the package, release, and support entitlement. The permission-prompting experiment asks the user to adjudicate requests — a cognitive surface, not a kernel boundary — and its defaults still require review.

A pragmatic supported baseline for workstation work, provided confinement mode and support entitlement are checked per application rather than assumed from the packaging name. The 26.04 direction moves Ubuntu toward mediated permissions, with the mediation burden landing on the operator.

Kicksecure

Kicksecure documents Debian-based hardening and a user/system-maintenance split with different boot roles; the split’s documentation labels verified boot as planned rather than shipped, with a conditional design for validating the boot chain before granting it trust (K. Project 2026a, 2026b, 2026c).

The hardening wiki explicitly contains research material, non-default proposals, and changes that can cause breakage; a long wiki page must not be mistaken for a list of shipped defaults. Verified boot remains a roadmap item, not an installed property.

A serious candidate where reducing everyday administrative authority is valuable. Judge the installed release and enabled settings; the split is an interesting direction for separating daily use from system maintenance, with boot integrity still on the roadmap rather than in the release.

Conventional Desktops: Hardening Candidates and Compatibility Costs IV

openSUSE Aeon

Aeon describes an immutable desktop based on Tumbleweed, while transactional-update prepares changes in a new snapshot without modifying the running system; the project still labels its releases release-candidate (openSUSE 2026a, 2026b).

Snapshot-based system replacement is not evidence that a compromised application cannot read or modify its authorized user data; the documented mechanism principally concerns system updates. The release-candidate label is itself a caveat an operator should not read past.

A credible image/snapshot-oriented desktop direction that has not completed its own stability review. This review did not verify role-specific defaults for the closely related MicroOS server image, and records those fields as n.a. rather than inferring them from Aeon's design.

Conventional Desktops: Hardening Candidates and Compatibility Costs I

Two honesty notes apply to the table. First, the Aeon entry deliberately withholds claims about the openSUSE MicroOS server image: this review did not verify all current MicroOS role-specific defaults or a distinct boot-integrity baseline, and the analysis treats those fields as n.a. rather than inferring server behavior from a desktop sibling that shares the transactional-update mechanism (openSUSE 2026b). Second, “immutable” is treated throughout as a statement about particular system paths and update mechanisms, never as a guarantee against runtime data theft from a compromised application.

Why secureblue is interesting, without crowning it I

secureblue's distinguishing proposition is not that the system is image-based. Its documented feature set combines allocator hardening through `hardened_malloc`, a specifically confined browser, removal of legacy privilege paths such as SUID-root programs, restricted desktop features, and efforts to retain supported sandbox use while limiting other namespace access (secureblue Project 2026a). That combination addresses more of the runtime attack surface than immutability alone, because it hardens the memory-error exploitation path and the browser — the two components most exposed to hostile content — rather than only the update mechanism. For a user who needs a recognizable conventional Linux desktop rather than the compartmentalized workflow of sec. 5, it is the candidate that most directly engages the exploitation failure path defined in

Why secureblue is interesting, without crowning it I

sec. 3.

The release history shows a specific engineering thesis, not a pile of toggles. Before v4.3.0, the project faced a genuine dilemma it documents itself: permit unprivileged user namespaces and leave a known privilege-escalation surface open, or disable them and run Flatpak and the browser through SUID-root helper binaries — a privilege path secureblue was simultaneously trying to eliminate (secureblue Project 2026b). The v4.3.0 resolution moves the decision from file-system bits into SELinux policy: unconfined processes and the container domain are denied user namespaces by default, while Flatpak and the browser run in their own SELinux domains that are granted namespace creation. The design is in the same family as Ubuntu's restricted unprivileged user namespaces introduced with Ubuntu 23.10, but stricter in the dimensions the project chooses to enforce (secureblue Project 2026b). The v4.4.0

Why secureblue is interesting, without crowning it I

cycle rebranded the hardened Chromium build as Trivalent — avoiding another project's trademark while allowing side-by-side installation with Fedora's stock Chromium (secureblue Project 2026a) — and the v4.9.1 release completed the removal of SUID-root programs, closing the legacy setuid privilege path the earlier policy work had made redundant. The direction matters more than any single item: hardening is migrating from user-configurable exceptions into shipped, policy-enforced defaults, which is exactly the shape the `application_confinement` property rewards.

Why secureblue is interesting, without crowning it I

The recommendation remains conditional for three documented reasons. The feature list does not establish independent resistance to a capable adversary: this review produced no comparative exploit-success or patch-latency measurements, and project documentation is not a substitute for either. The compatibility restrictions are real — disabled Xwayland and default-deny user namespaces will require exceptions for some workflows, and each exception is a hand-carved reduction of the baseline (secureblue Project 2026b). And the strongest hardening is reversible by the very operator it protects: a hardening menu that users routinely unwind under compatibility pressure fails the `human_usability` property regardless of its design quality. The conditional form matters more than the candidate's name: if secureblue's supported workflow fits the owner's applications, it can be preferable to assembling a large custom hardening stack; if exceptions proliferate,

Why secureblue is interesting, without crowning it I

a more standard environment that is actually maintained may deliver more security than an abandoned hardened one.

The Flatpak permission-grants warning I

Flatpak's documentation describes a restrictive basic sandbox, but also permission grants that expand access and explicitly warns about unrestricted bus access on the session or system D-Bus (Flatpak Project 2026). The practical consequence cuts against two common shorthand judgments. Neither “uses Flatpak” nor “has SELinux” is evidence that a particular application is confined to anything the operator would approve: a permission grant can hand an application access to files, devices, or the entire message bus, and the grant was usually made once, at install time, in exchange for the application running at all. The question that matters — which files, devices, services, and credentials this specific application can reach — is answered by inspecting the deployed grants, not by the packaging technology's reputation. This is complete mediation applied to application

The Flatpak permission-grants warning I

installation (Saltzer and Schroeder 1975): every expansion of authority should pass a checked interface, and the default answer should be the narrow one. Two developments in this review's candidate set move in that direction: secureblue's SELinux-scoped domain grants for Flatpak replace a blanket rule with a per-domain checked interface (secureblue Project 2026b), and Ubuntu 26.04's experimental snap permission prompting moves the grant decision from install time to use time (Canonical 2026). Both remain init systems' answers, not agent-era answers: neither mediates what an authorized agent may do with the access it holds.

The desktop-versus-agent authority lesson I

Every candidate in tbl. 1 optimizes the desktop's own attack surface: how its browser, allocator, update path, and application sandbox resist exploitation. None of them, as documented, answers the authorized-misuse path. A shell agent running on even the best-hardened desktop in this table holds whatever credentials and file access the operator's daily environment holds; allocator hardening and confined browsers do not mediate what an autonomous agent with a cloud token may read, transmit, or change. The lesson generalizes across the whole candidate set: a hardened desktop is a platform for trust boundaries, not a substitute for drawing them around autonomous agents. Agent work with untrusted dependencies belongs in a separate trust domain — a disposable VM, a separate container host, or a separate machine — with

The desktop-versus-agent authority lesson I

the authority architecture of sec. 10 governing what crosses that boundary. The property vocabulary of sec. 4 makes the distinction exact: `application_confinement` constrains applications; `authority` constrains what a component may already do without exploiting anything. A desktop can be strong on the first and still grant an agent catastrophic authority under the second. Scenario-level guidance for choosing among these candidates, including the conditions that would change the secureblue-first ordering, appears in sec. 16.